

Active Tremor Cancellation System

via Sensor Fusion and FES

Architecture and Feasibility Report

1 Problem Statement

Involuntary tremors, such as those caused by Essential Tremor or Parkinson's Disease, severely impact the daily lives of patients, making simple tasks like drinking water or writing incredibly difficult. Currently, treatments rely heavily on medications (which often cause systemic side effects like extreme fatigue or cognitive dulling) or highly invasive brain surgeries, such as Deep Brain Stimulation (DBS). There is a critical need for a wearable, non-invasive device that can automatically sense a tremor and physically stop it, without getting in the way of the person's normal, voluntary movements.

2 Proposed Solution

This project introduces a non-invasive, **closed-loop neuromodulation system**. The concept is similar to how noise-canceling headphones work, but applied to human biomechanics.

1. **Sensor Fusion (Listening to the Body):** The system relies on two key sensors. An Electromyography (EMG) sensor reads the tiny electrical signals from the brain telling the muscle to contract. An Inertial Measurement Unit (IMU) tracks the physical motion, measuring the speed and angle of the limb.
2. **Machine Learning (The Brain of the System):** The raw sensor data is cleaned and fed into a trained artificial intelligence model. This algorithm constantly analyzes the movements to distinguish between a normal, voluntary action (like reaching for a cup) and an involuntary tremor (typically a fast, rhythmic oscillation).
3. **Active Cancellation (Stopping the Tremor):** When the AI detects a tremor, it immediately calculates the exact timing (phase) and strength of the shaking. It then triggers a Functional Electrical Stimulation (FES/TENS) module. This device sends a safe, mild electrical pulse to the *antagonist* muscle (the muscle that pulls in the opposite direction). This creates a mechanical counter-force that dampens and cancels out the tremor in real-time.

3 Reality Check & Key Engineering Challenges

Building a closed-loop system that interacts directly with the human body is highly complex. The following critical challenges must be addressed:

- **Latency (The Timing Problem):** Timing is everything. If the software takes too long to read the data, detect the tremor, and send the corrective pulse, the patient's arm will have already moved. Firing the electrical pulse at the wrong time could actually *amplify* the tremor instead of stopping it. The entire detect-and-react loop must happen in under 30 milliseconds.

- **Stimulation Artifact (Blinding the Sensors):** The EMG sensor listens for tiny, natural electrical signals (measured in millivolts). When the TENS module fires a corrective electrical pulse (which is much stronger), it creates a massive spike of electrical noise that temporarily “blinds” the EMG sensor. The software must implement “blanking”—meaning it intentionally ignores EMG data for a split second right after a shock is delivered.
- **Electrical Safety:** Because we are attaching electrical stimulators to a human body, safety is paramount. The Arduino microcontroller and all sensors **must** be powered by batteries. They must be completely isolated from mains power (wall outlets) during human testing to prevent any risk of electric shock.
- **Muscle Fatigue:** Human muscles get tired very quickly when stimulated artificially. The control algorithm must be highly efficient, triggering the FES pulses only when absolutely necessary and using the lowest possible electrical current.

4 Closed-Loop Data Flow Map

The following diagram illustrates the continuous data flow from the patient, through the processing pipeline, and back to the patient as a therapeutic pulse.



5 Project Directory Structure

The software is highly modular to decouple heavy visualization tasks from safety-critical control loops.

```
tremor_cancellation_project/
|
+-- README.md                # Project documentation
+-- requirements.txt          # Python dependencies
|
+-- hardware_src/
|   +-- closed_loop_firmware.ino # Reads sensors & triggers TENS/FES
|
+-- python_src/
|   +-- main.py                # System orchestrator
|   |
|   +-- acquisition/
|   |   +-- serial_io.py        # Bi-directional Serial Comm
|   |
|   +-- processing/
|   |   +-- emg_filter.py        # Bandpass/Notch filters, RMS envelope
|   |   +-- imu_filter.py        # Sensor fusion (Madgwick/Kalman)
|   |
|   +-- visualization/
|   |   +-- plot_emg.py          # Real-time scrolling plot
|   |   +-- plot_imu.py          # 3D visualization of limb orientation
|   |
|   +-- storage/
|   |   +-- data_logger.py       # Background thread saving synchronized data
|   |
|   +-- ml_model/
|   |   +-- train_model.py        # Offline script for training
|   |   +-- inference.py          # Real-time script predicting tremor presence
|   |   +-- saved_models/         # Contains .pkl/.h5 trained weights
|   |
|   +-- control/
|       +-- stim_calculator.py    # Logic determining FES parameters
```

6 System File Breakdown

To make the system fast and reliable, the code is split into specific, modular files. This prevents heavy tasks (like drawing 3D graphs) from slowing down the critical safety checks.

6.1 Hardware Component

closed_loop_firmware.ino: This is the program running on the Arduino. It has two jobs: it constantly gathers data from the EMG and IMU sensors to send to the computer, and it listens for emergency commands from the computer to trigger the TENS stimulation pads.

6.2 Core & IO (Python)

- **main.py:** This is the "boss" or orchestrator of the system. It starts the program, loads the AI model, and manages the background threads so everything runs simultaneously without freezing.
- **serial_io.py:** This file manages the USB connection between the computer and the Arduino. It reads the incoming sensor data and instantly passes any stimulation commands back to the hardware.

6.3 Processing & Control

- **emg_filter.py & imu_filter.py:** Raw sensor data is very noisy. These files act as digital filters. They remove background static (like interference from power lines) and convert the raw numbers into clean, usable data (like the exact angle of the patient's arm).
- **inference.py:** This is where the Machine Learning model lives. It looks at the clean data and makes a split-second decision: *Is this a tremor? (True or False)*.
- **stim_calculator.py:** If a tremor is detected, this file does the math. It calculates exactly when the arm will shake next and determines the precise millisecond to trigger the electrical counter-pulse to stop it.

6.4 Visualization & Storage

- **plot_emg.py & plot_imu.py:** These files draw the live graphs and 3D models on the screen so researchers can monitor the patient. They run completely separately from the core logic so they do not cause any lag.
- **data_logger.py:** This acts as a flight data recorder. It silently saves all the sensor data, AI predictions, and stimulation timestamps to a spreadsheet (CSV file) so the team can analyze the results after the experiment is over.